MANAV RACHNA INTERNATIONAL INSTITUTE OF RESEARCH & STUDIES

SCHOOL OF COMPUTER APPLICATIONS

PRACTICAL FILE

Data Structure and Algorithm

Academic Session: 2026–2027

SUBMITTED TO

Mr. Sachin Sharma

Faculty / Assistant Professor

School of Computer Applications

MRIIRS

SUBMITTED BY

Priyanshu Singh

Roll No.: 26MCAP101N037

Course: MCA

Semester: 1

Part A — Array programs

1. Insert a new element in an array

```
a = [10, 20, 30, 40, 50]
print("Array:", a)

pos = int(input("Enter position: "))
x = int(input("Enter element: "))

if pos >= 1 and pos <= len(a) + 1:
    a.insert(pos - 1, x)
    print("New array:", a)
else:
    print("Invalid position")
```

```
Array: [10, 20, 30, 40, 50]
Enter position: 3
Enter element: 35
New array: [10, 20, 35, 30, 40, 50]
```

2. Delete an element from an array

```
a = [10, 20, 30, 40, 50]
print("Array:", a)

pos = int(input("Enter position to
delete: "))

if pos >= 1 and pos <= len(a):
    a.pop(pos - 1)
    print("New array:", a)
else:
    print("Invalid position")
```

```
Array: [10, 20, 30, 40, 50]
Enter position to delete: 3
New array: [10, 20, 40, 50]
```

3. Find the product of two matrices

This example multiplies two 2 x 2 matrices.

```
a = [[1, 2], [3, 4]]
b = [[5, 6], [7, 8]]
c = [[0, 0], [0, 0]]

for i in range(2):
    for j in range(2):
        for k in range(2):
            c[i][j] = c[i][j] + a[i][k] * b[k][j]

print("Product of matrices:")
for row in c:
    print(row)
```

```
Product of matrices:
[19, 22]
[43, 50]
```

4. Sum of diagonal and non-diagonal elements

```
a = [[1, 2, 3],
     [4, 5, 6],
     [7, 8, 9]]

d = 0
nd = 0

for i in range(3):
    for j in range(3):
        if i == j:
            d = d + a[i][j]
        else:
            nd = nd + a[i][j]

print("Diagonal sum:", d)
print("Non-diagonal sum:", nd)
```

```
Diagonal sum: 15
Non-diagonal sum: 30
```

5. Marks of 5 students in 3 subjects and percentage

Assume each subject is out of 100 marks.

```
for i in range(1, 6):
    print("Student", i)
    total = 0

    for j in range(1, 4):
        marks = int(input("Enter marks: "))
        total = total + marks

    per = total / 300 * 100

    print("Total marks:", total)
    print("Percentage:", per, "%")
```

```
Student 1
Enter marks: 80
Enter marks: 95
Enter marks: 98
Total marks: 273
Percentage: 91.0 %
Student 2
Enter marks:
```

Part B — Stack programs

6.1 Stack operations — push, pop and display

```
stack = []
while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Display")
    print("4. Exit")
    ch = int(input("Enter choice: "))
    if ch == 1:
        x = int(input("Enter element: "))
        stack.append(x)
        print("Element added")
    elif ch == 2:
        if len(stack) == 0:
            print("Stack is empty")
        else:
            print("Deleted element:", stack.pop())
    elif ch == 3:
        print("Stack:", stack)
    elif ch == 4:
        break
    else:
        print("Wrong choice")
```

```
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 5
Element added

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
Stack: [5]

1. Push
2. Pop
3. Display
4. Exit
Enter choice:
```

6.2 Reverse a string using a stack

```
s = input("Enter a string: ")
stack = []

for ch in s:
    stack.append(ch)

rev = ""

while len(stack) > 0:
    rev = rev + stack.pop()

print("Reversed string:", rev)
```

```
Enter a string: priyanshu
Reversed string: uhsnayirp
```

6.3 Parenthesis matching using a stack

This checks whether (), [] and {} are correctly matched.

```
s = input("Enter expression: ")
stack = []
valid = True
for ch in s:
    if ch == '(' or ch == '[' or ch == '{':
        stack.append(ch)
    elif ch == ')' or ch == ']' or ch == '}':
        if len(stack) == 0:
            valid = False
            break
        x = stack.pop()
        if (ch == ')' and x != '(') or \
           (ch == ']' and x != '[') or \
           (ch == '}' and x != '{'):
            valid = False
            break
if len(stack) != 0:
    valid = False
if valid:
    print("Parentheses are balanced")
else:
    print("Parentheses are not balanced")
```

```
Enter expression: [({})]
Parentheses are balanced
```

Part C — Queue programs

7.1 General queue operations — enqueue, dequeue and dis

```
q = []
while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")
    ch = int(input("Enter choice: "))
    if ch == 1:
        x = int(input("Enter element: "))
        q.append(x)
        print("Element added")
    elif ch == 2:
        if len(q) == 0:
            print("Queue is empty")
        else:
            print("Deleted element:", q.pop(0))
    elif ch == 3:
        print("Queue:", q)
    elif ch == 4:
        break
    else:
        print("Wrong choice")
```

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter element: 5
Element added

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 3
Queue: [5]

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice:
```

7.2 Circular queue operations

This implementation uses a fixed-size list and front and rear pointers, which is useful for demonstrating an actual circular queue.

```
size = 5
q = [0] * size
front = -1
rear = -1
while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")
    ch = int(input("Enter choice: "))
    if ch == 1:
        x = int(input("Enter element: "))
        if (rear + 1) % size == front:
            print("Queue is full")
        else:
            if front == -1:
                front = 0
            rear = (rear + 1) % size
            q[rear] = x
            print("Element added")
    elif ch == 2:
        if front == -1:
            print("Queue is empty")
```

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter element: 16
Element added

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 3
16

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice:
```

```
            else:
                print("Deleted element:", q[front])
                if front == rear:
                    front = -1
                    rear = -1
                else:
                    front = (front + 1) % size
    elif ch == 3:
        if front == -1:
            print("Queue is empty")
        else:
            i = front
            while True:
                print(q[i], end=" ")
                if i == rear:
                    break
                i = (i + 1) % size
            print()
    elif ch == 4:
        break
    else:
        print("Wrong choice")
```